



EXVUL

SMART CONTRACT **AUDIT REPORT**

Yala Vault Smart Contract

DECEMBER 2025

Contents

1. EXECUTIVE SUMMARY	4
1.1 Methodology	4
2. FINDINGS OVERVIEW	7
2.1 Project Info And Contract Address	7
2.2 Summary	7
2.3 Key Findings	8
3. DETAILED DESCRIPTION OF FINDINGS	9
3.1 01-Low-Reward depletion under partial payout clears remaining unclaimed rewards . . .	9
3.2 02-Low-Withdraw calculations carry the risk of underflow	11
3.3 03-Info-Comments explaining the error	14
4. CONCLUSION	16
5. APPENDIX	17
5.1 Basic Coding Assessment	17
5.1.1 Apply Verification Control	17
5.1.2 Authorization Access Control	17
5.1.3 Forged Transfer Vulnerability	17
5.1.4 Transaction Rollback Attack	18
5.1.5 Transaction Block Stuffing Attack	18
5.1.6 Soft Fail Attack Assessment	18
5.1.7 Hard Fail Attack Assessment	19
5.1.8 Abnormal Memo Assessment	19
5.1.9 Abnormal Resource Consumption	19
5.1.10 Random Number Security	20
5.2 Advanced Code Scrutiny	20
5.2.1 Cryptography Security	20
5.2.2 Account Permission Control	20
5.2.3 Malicious Code Behavior	21
5.2.4 Sensitive Information Disclosure	21
5.2.5 System API	21
6. DISCLAIMER	22
7. REFERENCES	23

8. About Exvul Security

24

1. EXECUTIVE SUMMARY

ExVul Web3 Security was engaged by **Yala Vault** to review smart contract implementation. The assessment was conducted in accordance with our systematic approach to evaluate potential security issues based upon customer requirement. The report provides detailed recommendations to resolve the issue and provide additional suggestions or recommendations for improvement.

The outcome of the assessment outlined in chapter 3 provides the system's owners a full description of the vulnerabilities identified, the associated risk rating for each vulnerability, and detailed recommendations that will resolve the underlying technical issue.

1.1 Methodology

To standardize the evaluation, we define the following terminology based on OWASP Risk Rating Methodology [10] which is the gold standard in risk assessment using the following risk models:

- **Likelihood:** represents how likely a particular vulnerability is to be uncovered and exploited in the wild.
- **Impact:** measures the technical loss and business damage of a successful attack.
- **Severity:** determine the overall criticality of the risk.

Likelihood can be: High, Medium and Low and impact are categorized into: High, Medium, Low, Informational. Severity is determined by likelihood and impact and can be classified into five categories accordingly: Critical, High, Medium, Low, Informational shown in table 1.1.

		Informational	Low	Medium	High
Likelihood	High	INFO	MEDIUM	HIGH	CRITICAL
	Medium	INFO	LOW	MEDIUM	HIGH
	Low	INFO	LOW	LOW	MEDIUM
		IMPACT			

Table 1.1 Overall Risk Severity

To evaluate the risk, we will be going through a list of items, and each would be labelled with a severity category. The audit was performed with a systematic approach guided by a comprehensive assessment list carefully designed to identify known and impactful security issues. If our tool or analysis does not identify any issue, the contract can be considered safe regarding the assessed item. For any discovered issue, we might further deploy contracts on our private test environment and run tests to confirm the findings. If necessary, we would additionally build a PoC to demonstrate the possibility of exploitation. The concrete list of check items is shown in Table 1.2.

- **Basic Coding Bugs:** We first statically analyze given smart contracts with our proprietary static code analyzer for known coding bugs, and then manually verify (reject or confirm) all the issues found by our tool.
- **Code and business security testing:** We further review business logics, examine system operations, and place DeFi-related aspects under scrutiny to uncover possible pitfalls and/or bugs.
- **Additional Recommendations:** We also provide additional suggestions regarding the coding and development of smart contracts from the perspective of proven programming practices.

Category	Assessment Item
Basic Coding Assessment	• Apply Verification Control • Authorization Access Control • Forged Transfer Vulnerability • Forged Transfer Notification • Numeric Overflow • Transaction Rollback Attack • Transaction Block Stuffing Attack • Soft Fail Attack • Hard Fail Attack • Abnormal Memo • Abnormal Resource Consumption • Secure Random Number

Advanced Source Code Scrutiny	• Asset Security • Cryptography Security • Business Logic Review • Source Code Functional Verification • Account Authorization Control • Sensitive Information Disclosure • Circuit Breaker • Blacklist Control • System API Call Analysis • Contract Deployment Consistency Check • Abnormal Resource Consumption
Additional Recommendations	• Semantic Consistency Checks • Following Other Best Practices

Table 1.2: The Full List of Assessment Items

To better describe each issue we identified, we categorize the findings with Common Weakness Enumeration (CWE-699) [14], which is a community-developed list of software weakness types to better delineate and organize weaknesses around concepts frequently encountered in software development.

2. FINDINGS OVERVIEW

2.1 Project Info And Contract Address

Project Name	Audit Time	Language
Yala Vault	8/12/2025 - 9/12/2025	Solidity

Repository

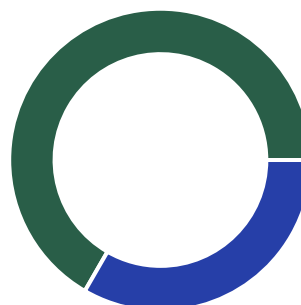
<https://github.com/yalaorg/yala-vault>

Commit Hash

9f524fab5d6b0a947646af17731b275861b2e754

2.2 Summary

Severity	Found
CRITICAL	0
HIGH	0
MEDIUM	0
LOW	2
INFO	1



2.3 Key Findings

Severity	Findings Title	Status
LOW	01-Low-Reward depletion under partial payout clears remaining unclaimed rewards	Fixed
LOW	02-Low-Withdraw calculations carry the risk of underflow	Fixed
INFO	03-Info-Comments explaining the error	Fixed

Table 2.3: Key Audit Findings

3. DETAILED DESCRIPTION OF FINDINGS

3.1 01-Low-Reward depletion under partial payout clears remaining unclaimed rewards

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/core/YalaVault.sol

DESCRIPTION:

claimGains collapses the full logical reward debt into a single “all-or-nothing” variable reset, instead of decrementing the user’s reward accounting by the portion that was actually paid. This design assumes that rewardToken.balanceOf(address(this)) is always sufficient to cover _gains. Once this assumption is violated (e.g. administrative withdrawals of rewardToken, or accounting mismatch introduced elsewhere), the internal reward debt and the real token reserves diverge and the shortfall is irreversibly pushed onto the user who happens to claim after the vault becomes undercollateralized.

```
/// @dev claim gains for a specific token index
/// @param index The index of the token to claim gains from (0 or 1)
function claimGains(uint8 index) internal validIndex(index) nonReentrant {
    _accrueRewardGains(index, msg.sender);
    AccountInfo storage deposits = accountInfos[msg.sender];
    uint256 initialShares = deposits.shares[index];
    uint256 newShares = _getCompoundedShares(index, msg.sender,
        initialShares);
    deposits.shares[index] = newShares;
    _updateSnapshots(index, deposits, newShares);
    uint256 _gains = _computeAsset(rewardToken, gains[msg.sender][index]);
    uint256 claimable = Math.min(_gains,
        rewardToken.balanceOf(address(this)));
    if (claimable > 0) {
        gains[msg.sender][index] = 0;
        IERC20(rewardToken).safeTransfer(msg.sender, claimable);
        emit ClaimGains(msg.sender, index, claimable);
    }
}
```

IMPACT:

When the vault's actual rewardToken reserves are lower than the mathematically accrued rewards, a depositor who calls claimGains will: - receive only the truncated claimable amount - lose the residual reward entitlement forever, because gains[msg.sender][index] is reset to zero regardless of the payout ratio.

RECOMMENDATIONS:

Align the internal reward accounting with the actually transferred payout by decrementing the stored reward shares proportionally to the amount paid, or enforce that the vault is fully collateralized before allowing claims.

```
/// @dev claim gains for a specific token index
/// @param index The index of the token to claim gains from (0 or 1)
function claimGains(uint8 index) internal validIndex(index) nonReentrant {
    _accrueRewardGains(index, msg.sender);
    AccountInfo storage deposits = accountInfos[msg.sender];
    uint256 initialShares = deposits.shares[index];
    uint256 newShares = _getCompoundedShares(index, msg.sender,
initialShares);
    deposits.shares[index] = newShares;
    _updateSnapshots(index, deposits, newShares);
    uint256 _gains = _computeAsset(rewardToken, gains[msg.sender][index]);
+       require(
+           _gains <= rewardToken.balanceOf(address(this)),
+           'YalaVault: insufficient reward balance'
+       );
-       uint256 claimable = Math.min(_gains,
rewardToken.balanceOf(address(this)));
+       uint256 claimable = _gains;
    if (claimable > 0) {
        gains[msg.sender][index] = 0;
        IERC20(rewardToken).safeTransfer(msg.sender, claimable);
        emit ClaimGains(msg.sender, index, claimable);
    }
}
```

3.2 02-Low-Withdraw calculations carry the risk of underflow

SEVERITY:

LOW

STATUS:

Fixed

PATH:

contracts/core/YalaVault.sol

DESCRIPTION:

In the deposit function, users can freely deposit assets greater than 0, which are then converted into shares and counted by `totalShares[index]`.

```
function deposit(uint8 index, uint256 _amount) external validIndex(index)
    nonReentrant {
        require(_amount > 0, 'YalaVault: deposit amount must be greater than
        zero');

        // ...

        uint256 shares = _computeAssetShares(tokens[index], _amount);
        totalShares[index] += shares;

        // ...
    }
```

However, during the withdraw process, `totalShares[index] - MIN_REMAIN` is calculated. If `totalShares[index] < MIN_REMAIN`, a revert will occur, causing the withdrawal operation to fail.

```
function withdraw(uint8 index, uint256 _shares) external
    validIndex(index) nonReentrant {
        require(_shares > 0, 'YalaVault: withdraw amount must be greater than
        zero');

        // ...

        uint256 maxWithdrawal = Math.min(_shares, compoundedShares);
        uint256 withdrawable = Math.min(maxWithdrawal, totalShares[index] -
        MIN_REMAIN);
        uint256 newShares = compoundedShares - withdrawable;
```

```
// ...  
}
```

IMPACT:

When `totalShares[index]` is insufficient, the withdraw operation will revert without returning accurate error information.

RECOMMENDATIONS:

When the withdraw function is executed, the sizes of `totalShares[index]` and `MIN_REMAIN` are checked and processed accordingly.

```
function withdraw(uint8 index, uint256 _shares) external  
    validIndex(index) nonReentrant {  
        require(_shares > 0, 'YalaVault: withdraw amount must be greater than  
zero');  
        AccountInfo storage deposits = accountInfos[msg.sender];  
        uint256 initialShares = deposits.shares[index];  
        _accrueRewardGains(index, msg.sender);  
        uint256 compoundedShares = _getCompoundedShares(index, msg.sender,  
initialShares);  
        uint256 maxWithdrawal = Math.min(_shares, compoundedShares);  
-    uint256 withdrawable = Math.min(maxWithdrawal, totalShares[index] -  
MIN_REMAIN);  
+    uint256 withdrawable;  
+    if (totalShares[index] >= MIN_REMAIN) {  
+        withdrawable = Math.min(maxWithdrawal, totalShares[index] -  
MIN_REMAIN);  
+    } else {  
+        revert('YalaVault: pool too small to withdraw');  
+    }  
        uint256 newShares = compoundedShares - withdrawable;  
        deposits.shares[index] = newShares;  
        if (withdrawable > 0) {  
            IERC20(tokens[index]).safeTransfer(msg.sender,  
_computeAsset(tokens[index], withdrawable));  
            totalShares[index] -= withdrawable;  
        }  
        _updateSnapshots(index, deposits, newShares);
```

```
    emit Withdraw(msg.sender, index, newShares, _shares);  
}
```

3.3 03-Info-Comments explaining the error

SEVERITY:

INFO

STATUS:

Fixed

PATH:

contracts/core/YalaVault.sol

DESCRIPTION:

The getCompoundedShares function returns a compoundedShares value, but the comments describe it as gains.

```
/// @dev get compounded shares for a depositor
/// @param index The index of the token
/// @param _depositor The address of the depositor
/// @return The total gains of the depositor
function getCompoundedShares(uint8 index, address _depositor) public view
    returns (uint256) {
    ...
}
```

IMPACT:

The discrepancy between the comment and the return value can easily lead to ambiguity.

RECOMMENDATIONS:

Repair documentation instructions, /// @return The total compoundedShares of the depositor.

```
/// @dev get compounded shares for a depositor
/// @param index The index of the token
/// @param _depositor The address of the depositor
-/// @return The total gains of the depositor
+/// @return The total compoundedShares of the depositor
function getCompoundedShares(uint8 index, address _depositor) public view
    returns (uint256) {
    ...
}
```

}

4. CONCLUSION

In this audit, we thoroughly analyzed **Yala Vault** smart contract implementation. The problems found are described and explained in detail in Section 3. The problems found in the audit have been communicated to the project leader. We therefore consider the audit result to be **PASSED**.

To improve this report, we greatly appreciate any constructive feedbacks or suggestions, on our methodology, audit findings, or potential gaps in scope/coverage.

5. APPENDIX

5.1 Basic Coding Assessment

5.1.1 Apply Verification Control

Description	The security of apply verification
Result	Not found
Severity	CRITICAL

5.1.2 Authorization Access Control

Description	Permission checks for external integral functions
Result	Not found
Severity	CRITICAL

5.1.3 Forged Transfer Vulnerability

Description	Assess whether there is a forged transfer notification vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.4 Transaction Rollback Attack

Description	Assess whether there is transaction rollback attack vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.5 Transaction Block Stuffing Attack

Description	Assess whether there is transaction blocking attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.6 Soft Fail Attack Assessment

Description	Assess whether there is soft fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.7 Hard Fail Attack Assessment

Description	Examine for hard fail attack vulnerability
Result	Not found
Severity	CRITICAL

5.1.8 Abnormal Memo Assessment

Description	Assess whether there is abnormal memo vulnerability in the contract
Result	Not found
Severity	CRITICAL

5.1.9 Abnormal Resource Consumption

Description	Examine whether abnormal resource consumption in contract processing
Result	Not found
Severity	CRITICAL

5.1.10 Random Number Security

Description	Examine whether the code uses insecure random number
Result	Not found
Severity	CRITICAL

5.2 Advanced Code Scrutiny

5.2.1 Cryptography Security

Description	Examine for weakness in cryptograph implementation
Result	Not found
Severity	HIGH

5.2.2 Account Permission Control

Description	Examine permission control issue in the contract
Result	Not found
Severity	MEDIUM

5.2.3 Malicious Code Behavior

Description	Examine whether sensitive behavior present in the code
Result	Not found
Severity	MEDIUM

5.2.4 Sensitive Information Disclosure

Description	Examine whether sensitive information disclosure issue present in the code
Result	Not found
Severity	MEDIUM

5.2.5 System API

Description	Examine whether system API application issue present in the code
Result	Not found
Severity	LOW

6. DISCLAIMER

This report is subject to the terms and conditions (including without limitation, description of services, confidentiality, disclaimer and limitation of liability) set forth in the Services Agreement, or the scope of services, and terms and conditions provided to the Company in connection with the Agreement. This report provided in connection with the Services set forth in the Agreement shall be used by the Company only to the extent permitted under the terms and conditions set forth in the Agreement. This report may not be transmitted, disclosed, referred to or relied upon by any person for any purposes without ExVul's prior written consent.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts ExVul to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. ExVul's position is that each company and individual are responsible for their own due diligence and continuous security. ExVul's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

7. REFERENCES

- [1] MITRE. CWE-191: Integer Underflow (Wrap or Wraparound). <https://cwe.mitre.org/data/definitions/191.html>.
 - [2] MITRE. CWE-197: Numeric Truncation Error. <https://cwe.mitre.org/data/definitions/197.html>.
 - [3] MITRE. CWE-400: Uncontrolled Resource Consumption. <https://cwe.mitre.org/data/definitions/400.html>.
 - [4] MITRE. CWE-440: Expected Behavior Violation. <https://cwe.mitre.org/data/definitions/440.html>.
 - [5] MITRE. CWE-684: Protection Mechanism Failure. <https://cwe.mitre.org/data/definitions/693.html>.
 - [6] MITRE. CWE CATEGORY: 7PK - Security Features. <https://cwe.mitre.org/data/definitions/254.html>.
 - [7] MITRE. CWE CATEGORY: Behavioral Problems. <https://cwe.mitre.org/data/definitions/438.html>.
 - [8] MITRE. CWE CATEGORY: Numeric Errors. <https://cwe.mitre.org/data/definitions/189.html>.
 - [9] MITRE. CWE CATEGORY: Resource Management Errors. <https://cwe.mitre.org/data/definitions/399.html>.
 - [10] OWASP. Risk Rating Methodology. https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology
-

8. About Exvul Security

Premier Security for the Web3 Ecosystem

ExVul is a premier Web3 security firm committed to forging a secure and trustworthy decentralized ecosystem. Our elite team consists of security veterans from world-leading technology and blockchain security firms, including Huawei, YBB Captical, Qihoo 360, Amber, ByteDance, MoveBit, and PeckShield. Team member Nolan is ranked as a top-40 whitehat on Immunefi and is the platform's sole All-Star in the APAC region.

Our expertise covers the full spectrum of Web3 security. We conduct **meticulous smart contract audits**, having fortified thousands of projects on chains like Evm, Solana, Aptos, Sui etc. Our **Blockchain Protocol Audits** secure the core infrastructure of L1/L2 by uncovering deep-seated vulnerabilities. We also offer **comprehensive wallet audits** to protect user assets and provide **proactive web3 pentest**, enabling partners to neutralize threats before they strike.

Trusted by industry leaders, ExVul is the security partner for **OKX, Bitget, Cobo, Infini, Stacks, Aptos, Sui, CoreDAO, Sei** etc.

Contact

 **Website**
www.exvul.com

 **Twitter**
[@EXVULSEC](https://twitter.com/EXVULSEC)

 **Email**
contact@exvul.com

 **Github**
github.com/EXVUL-Sec